# 11 Reporting Service

Enterprise VAT Management Platform | ASP.NET Core + PostgreSQL + RabbitMQ

## ADVANCED BACKEND IMPLEMENTATION SPECIFICATION

Document status: Reference architecture / implementation-ready design guidance. Regulatory mapping and integration contracts require validation against authoritative requirements.

## 1. Purpose and bounded context

Read models, dashboards, scheduled exports and audit reconciliation The service owns its database and domain invariants. It exposes versioned HTTP APIs and publishes integration events through a transactional outbox. No other service directly accesses its tables.

## 2. Service contract and ownership

Service name: ReportingService. Database: vat_reporting_db. Deployable: independent ASP.NET Core Web API + background worker. Ownership: API contract, schema migrations, events, alerts, security policies, observability and operational runbooks.

## 3. Domain entities and tables

Proposed tables: report_projections, report_jobs, report_templates, report_exports, projection_checkpoints, reconciliation_alerts. Each write model includes id, company_id (where relevant), created_at, updated_at, created_by, row_version; posting records additionally include source reference, business date, correlation_id and reversal linkage.

## 4. REST API specification

Version prefix /api/v1. Proposed endpoints:
GET /reports/dashboard
GET /reports/purchases
GET /reports/sales
POST /reports/exports
GET /reports/jobs/{id}
All mutating operations accept Idempotency-Key where applicable; use ProblemDetails for errors, OpenAPI contracts, cursor/page-based listing and tenant-scoped filtering.

## 5. Events and messaging

Integration events: ReportProjectionUpdatedV1, ReportExportCompletedV1, ReconciliationAlertRaisedV1. Event envelope: eventId UUID, eventType, schemaVersion, occurredAt UTC, correlationId, causationId, companyId, branchId where applicable, aggregateId, aggregateVersion and payload. Outbox messages commit with local writes. Consumers use durable inbox deduplication.

## 6. Business rules and consistency

CQRS projections are eventually consistent; expose watermark and last refreshed timestamp. Commands must validate entity state transitions and reference snapshots. Use optimistic concurrency and enforce uniqueness in PostgreSQL. External effects are retried and reconciled; no distributed ACID assumptions.

## 7. Security and audit

Row-level company filters, signed expiring download links and export audit. Validate access token audience/issuer, explicit permission, company/branch scope, resource ownership and current document state inside the service. Log actor, before/after metadata (redacted), reason and correlation ID.

## 8. Reference implementation structure

services/Reporting/
Reporting.Api/ (Endpoints, auth, OpenAPI, middleware)
Reporting.Application/ (Commands, queries, validation, handlers)
Reporting.Domain/ (Aggregates, rules, events)
Reporting.Infrastructure/ (EF Core, migrations, outbox, inbox)
Reporting.Worker/ (event consumers, reconciliation jobs)
Reporting.Tests/ (unit, integration, contract, security)

## 9. Representative API request / response

POST /api/v1/reporting/commands
Headers: Authorization: Bearer <access-token>; Idempotency-Key: <uuid>; X-Correlation-ID: <uuid>
Request: { "companyId": 101, "branchId": 5, "externalReference": "ERP-2026-001", "data": {} }
Response: 202 Accepted { "operationId": "uuid", "status": "Pending", "statusUrl": "/api/v1/operations/uuid" }
Note: illustrative command envelope, not a universal endpoint implemented by every service.

## 10. Transactional outbox pseudocode

BEGIN DATABASE TRANSACTION
Validate actor, company, workflow status, version and domain rules
Apply aggregate state change
INSERT outbox_messages(event_id, event_type, payload, created_at)
COMMIT
Outbox dispatcher publishes to RabbitMQ with publisher confirmation.
Consumer: INSERT inbox_messages(event_id) ON CONFLICT DO NOTHING; only first insert applies business change in the same transaction.

## 11. PostgreSQL schema principles

CREATE DATABASE vat_reporting_db;
-- Apply EF Core migrations owned only by ReportingService.
-- Include unique(company_id, external_reference) where meaningful.
-- Monetary amounts NUMERIC(20,4), UTC timestamps TIMESTAMPTZ.
-- Add indexes for company_id, business_date, status, correlation_id.
-- Partition high-volume ledger/event tables only after measured need.

## 12. Operational design

Liveness and readiness probes, OpenTelemetry traces, structured logs, p95 latency and queue-lag metrics, consumer retry and dead-letter alerting, backup/PITR and restore drills. Health checks must not leak credentials. Use per-service database credentials and least-privilege grants.

## 13. Failure scenarios and recovery

Database unavailable: fail closed for writes, retry with bounded backoff. RabbitMQ unavailable: commit outbox locally, publish later. Duplicate event: inbox rejects duplicate. Downstream timeout: query operation status and reconcile. Irreversible posted records: compensate with authorized reversal rather than deletion.

## 14. Verification matrix

Unit: domain state transitions, monetary rounding and invariants. Integration: PostgreSQL migrations, isolation, concurrency and unique constraints. Contract: OpenAPI and event schema compatibility. End-to-end: create -> approve -> publish -> consume -> reconcile. Security: cross-company IDOR, role escalation, self-approval, token expiry. Resilience: broker outage, replay, poison event and partial Saga failure.

## 15. Delivery acceptance criteria

1. ReportingService builds and deploys independently.
2. OpenAPI is reviewed and versioned.
3. EF Core migrations run against isolated PostgreSQL database.
4. Authorization is enforced inside service and negative-tested.
5. Transactional outbox/inbox verified with duplicate and broker-failure tests.
6. Events have stable contracts and schema versions.
7. Traces and audit logs show end-to-end correlation.
8. Reconciliation/repair runbook and rollback strategy are documented.

## 16. Recommended implementation sequence

Sprint A: domain model, migrations and OpenAPI. Sprint B: command/query handlers and RBAC. Sprint C: outbox, inbox, consumers and idempotency. Sprint D: observability, contract tests, failure injection and deployment. Integrate with Angular through YARP, with no frontend direct access to internal service ports.

## 17. Reference quality gates

Use official .NET/ASP.NET Core, EF Core, PostgreSQL, RabbitMQ and MassTransit documentation for exact library APIs and supported versions. For VAT forms, rates and return rules, verify current NBR law, SROs and form specifications before certification or production use. These documents are original engineering blueprints, not source code or verified government certification.